Q1: Attempt any Five $(2\times5=10)$

a) Define Machine Learning.

Machine Learning (ML) is a field of artificial intelligence that involves training computers to learn from data and improve their performance on a task without being explicitly programmed[1]. It focuses on the **development of algorithms** that allow computers to automatically adjust their behavior based on new data[2].

b) List any four techniques for handling missing values from datasets.

Techniques for handling missing values include[333]:

1. **Imputation:** Filling in the missing values, often using the **mean, median, or mode** of the column[4].
2. **Dropping rows/columns:** Removing the rows (samples) or columns (features) that contain missing data[5].
3. **Prediction/Modeling:** Using a machine learning model to **predict** the missing values based on other features.
4. **Using a special value:** Replacing the missing value with a category like "Missing" for categorical data or a specific number like -999 for numerical data.

c) State Principal Component Analysis in ML.

Principal Component Analysis (PCA) is an unsupervised learning technique used for **dimensionality reduction**[6]. Its goal is to reduce the number of features (variables) in a dataset while retaining most of the variation, or information[77]. It does this by transforming the data into a new set of dimensions called **Principal Components**[88].

d) State Hyperparameter Tuning of Model.

Hyperparameter Tuning is the process of selecting the optimal set of **hyperparameters** for a machine learning algorithm[9]. Hyperparameters are external configuration settings (e.g., the learning rate in a neural network, or k in k-NN [10]) whose values cannot be estimated from the data. Tuning is done to improve the **model's performance** and generalization ability[11].

e) List four applications of ML in the Healthcare domain.

Applications of ML in the healthcare domain include[12]:

1. **Medical Diagnostics:** Assisting doctors in the diagnosis of diseases (e.g., classifying medical images)[13].
2. **Drug Discovery and Research:** Accelerating the process of finding new drugs.
3. **Predictive Analytics:** Predicting patient outcomes or disease outbreaks.
4. **Personalized Treatment:** Recommending treatments tailored to an individual patient's genetic and medical profile.

f) State the applications of the given libraries: Scikit-learn, Pandas, Numpy and Matplotlib

Library	Application/Use	Aligned CO
Scikit-learn	Used for implementing most **Machine Learning algorithms** (supervised and unsupervised) and for model evaluation[14141414].	CO1
Pandas	Used for **data loading, manipulation, and analysis**, providing structures like DataFrames[151515].	CO1
Numpy	Used for high-performance numerical computing, especially for **array and matrix operations**[1616].	CO1
Matplotlib	Used for **data visualization** to create static, animated, and interactive plots and graphs[17].	CO1

g) State Mean Squared Error and Mean Average Error with importance in ML.

- **Mean Squared Error (MSE):** The average of the **squared differences** between the predicted values and the actual values[18].
  - **Importance:** MSE heavily penalizes large errors because the errors are squared, making it a good metric to use when large errors are particularly undesirable[19]. It is one of the evaluation metrics used for **Linear Regression**[2020].
- **Mean Absolute Error (MAE):** The average of the **absolute differences** between the predicted values and the actual values.
  - **Importance:** MAE is less sensitive to outliers than MSE since it doesn't square the error. It provides a more robust measure of average error magnitude.

Q2: Attempt any Three $(4\times3=12)$

a) Compare Supervised with Unsupervised Learning.

Feature	Supervised Learning	Unsupervised Learning	Aligned CO
Definition	The model is trained on a **labeled dataset**, where both the input data and the corresponding output (label) are provided[21212121].	The model is trained on an **unlabeled dataset** and tries to find hidden patterns or structures on its own[22222222].	CO1
Goal	To **predict** the output for new, unseen input data.	To **discover** underlying structure, patterns, or groupings within the data.	CO1

Feature	Supervised Learning	Unsupervised Learning	Aligned CO
Data	Requires **labeled data**.	Requires **unlabeled data**.	CO1
Common Tasks	Classification and Regression[23][23].	Clustering and Dimensionality Reduction (e.g., K-Means, PCA)[24][24][24].	CO1
Example	Predicting house prices (Regression) or classifying emails as spam or not spam (Classification)[25].	Customer segmentation (Clustering)[26].	CO1

b) Describe Precision and Recall with formula.

Precision and **Recall** are model evaluation metrics particularly important for **Classification** problems, especially when the classes are imbalanced[27]. They are calculated from the **Confusion Matrix**[28].

- **Precision:**
  - **Description:** Precision answers the question: **"Out of all the instances the model *predicted* as positive, how many were *actually* positive?"**[29]. High precision minimizes **False Positives**.
  - Formula:

    $$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Recall (Sensitivity):**
  - **Description:** Recall answers the question: **"Out of all the instances that were *actually* positive, how many did the model *correctly identify*?"**[30]. High recall minimizes **False Negatives**.
  - Formula:

    $$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

c) Write a Python program to convert categorical columns (eg, "Male/Female") into numerical format using one-hot encoding and label encoding (Assume any tabular example)

Python
```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Assume a sample dataset
data = {
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
```

```
    'City': ['Mumbai', 'Delhi', 'Mumbai', 'Chennai', 'Delhi',
'Mumbai'],
    'Age': [25, 30, 45, 22, 35, 50]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# --- 1. Label Encoding ---
# Used for ordinal categorical data (where order matters) or for binary
features.
le = LabelEncoder()
df['Gender_Label_Encoded'] = le.fit_transform(df['Gender'])
print("\nDataFrame after Label Encoding on 'Gender':\n", df)

# --- 2. One-Hot Encoding ---
# Used for nominal categorical data (where order doesn't matter).
# We'll apply this to the 'City' column.
df_ohe = pd.get_dummies(df, columns=['City'], prefix='City')
print("\nDataFrame after One-Hot Encoding on 'City':\n", df_ohe)

# Cleanup for final display
df_final = df_ohe.drop(columns=['Gender'])
print("\nFinal Result (Combined Encodings):\n", df_final)
```

d) Write a Program to cluster given data into two groups using K-Means clustering assuming initial centroid as $M_1=4$ and $M_2=11$. Data points $S=\{2,4,10,12,3,20,30,11,25\}$

This is a mathematical problem that can be solved step-by-step using the K-Means algorithm[31].

Data Points (S): $\{2, 4, 10, 12, 3, 20, 30, 11, 25\}$

Initial Centroids: $M_1 = 4$, $M_2 = 11$

Iteration 1: Assignment Step

Assign each data point to the nearest centroid using the **Euclidean distance** formula: $d(x, M) = |x - M|$.

| Data Point (x) | Distance to M_1 ($|x-4|$) | Distance to M_2 ($|x-11|$) | Cluster |

| :--- | :--- | :--- | :--- |

| 2 | $|2-4|=2$ | $|2-11|=9$ | Cluster 1 |

| 4 | $|4-4|=0$ | $|4-11|=7$ | Cluster 1 |

| 10 | $|10-4|=6$ | $|10-11|=1$ | Cluster 2 |

| 12 | $|12-4|=8$ | $|12-11|=1$ | Cluster 2 |

| 3 | $|3-4|=1$ | $|3-11|=8$ | Cluster 1 |

| 20 | $|20-4|=16$ | $|20-11|=9$ | Cluster 2 |

| 30 | $|30-4|=26$ | $|30-11|=19$ | Cluster 2 |

| 11 | $|11-4|=7$ | $|11-11|=0$ | Cluster 2 |

| 25 | $|25-4|=21$ | $|25-11|=14$ | Cluster 2 |

- **Cluster 1 (C1):** $\{2, 4, 3\}$
- **Cluster 2 (C2):** $\{10, 12, 20, 30, 11, 25\}$

Iteration 1: Update Step

Calculate the new centroid (mean) for each cluster.

- **New M_1:** $\frac{2 + 4 + 3}{3} = \frac{9}{3} = **3.0$**
- **New M_2:** $\frac{10 + 12 + 20 + 30 + 11 + 25}{6} = \frac{108}{6} = **18.0$**

Iteration 2: Assignment Step

New Centroids: $M_1 = 3.0$, $M_2 = 18.0$.

| Data Point (x) | Distance to M_1 ($|x-3.0|$) | Distance to M_2 ($|x-18.0|$) | Cluster |

| :--- | :--- | :--- | :--- |

| 2 | $|2-3.0|=1.0$ | $|2-18.0|=16.0$ | Cluster 1 |

| 4 | $|4-3.0|=1.0$ | $|4-18.0|=14.0$ | Cluster 1 |

| 10 | $|10-3.0|=7.0$ | $|10-18.0|=8.0$ | Cluster 1 |

| 12 | $|12-3.0|=9.0$ | $|12-18.0|=6.0$ | Cluster 2 |

| 3 | $|3-3.0|=0.0$ | $|3-18.0|=15.0$ | Cluster 1 |

| 20 | $|20-3.0|=17.0$ | $|20-18.0|=2.0$ | Cluster 2 |

| 30 | $|30-3.0|=27.0$ | $|30-18.0|=12.0$ | Cluster 2 |

| 11 | $|11-3.0|=8.0$ | $|11-18.0|=7.0$ | Cluster 2 |

| 25 | $|25-3.0|=22.0$ | $|25-18.0|=7.0$ | Cluster 2 |

- **Cluster 1 (C1):** $\{2, 4, 10, 3\}$
- **Cluster 2 (C2):** $\{12, 20, 30, 11, 25\}$

Iteration 2: Update Step

- **New M_1:** $\frac{2 + 4 + 10 + 3}{4} = \frac{19}{4} = **4.75$**

- **New M_2:** $\frac{12 + 20 + 30 + 11 + 25}{5} = \frac{98}{5} = **19.6$**

  Iteration 3: Assignment Step

  New Centroids: $M_1 = 4.75$, $M_2 = 19.6$.

  | Data Point (x) | Distance to M_1 ($|x-4.75|$) | Distance to M_2 ($|x-19.6|$) | Cluster |

  | :--- | :--- | :--- | :--- |

  | 2 | 2.75 | 17.6 | Cluster 1 |

  | 4 | 0.75 | 15.6 | Cluster 1 |

  | 10 | 5.25 | 9.6 | Cluster 1 |

  | 12 | 7.25 | 7.6 | Cluster 1 |

  | 3 | 1.75 | 16.6 | Cluster 1 |

  | 20 | 15.25 | 0.4 | Cluster 2 |

  | 30 | 25.25 | 10.4 | Cluster 2 |

  | 11 | 6.25 | 8.6 | Cluster 1 |

  | 25 | 20.25 | 5.4 | Cluster 2 |

- **Cluster 1 (C1):** $\{2, 4, 10, 12, 3, 11\}$
- **Cluster 2 (C2):** $\{20, 30, 25\}$

  Iteration 3: Update Step

- **New M_1:** $\frac{2 + 4 + 10 + 12 + 3 + 11}{6} = \frac{42}{6} = **7.0$**
- **New M_2:** $\frac{20 + 30 + 25}{3} = \frac{75}{3} = **25.0$**

  Since the cluster assignments changed again, the process would continue until the assignments no longer change.

  Final Clusters (as of Iteration 3):

- **Group 1:** $\{2, 3, 4, 10, 11, 12\}$
- **Group 2:** $\{20, 25, 30\}$

e) State PCA and K-Means clustering with example.

- **Principal Component Analysis (PCA):**

- **State:** PCA is a technique for **dimensionality reduction** which identifies new orthogonal axes (Principal Components) that capture the maximum variance in the data[32][32]. It is primarily used to simplify datasets while preserving important information[33].
- **Example:** Reducing a dataset with 100 features (e.g., spectral bands in satellite imagery) down to just 2 or 3 Principal Components for easier **visualization** and faster training of subsequent machine learning models[34].
- **K-Means Clustering:**
- **State:** K-Means is an **unsupervised clustering algorithm** that partitions N observations into K clusters[35]. It does this by iteratively assigning data points to the nearest cluster centroid and then re-calculating the centroid as the mean of the points in the cluster[36].
- **Example: Customer Segmentation**[37]. A retailer can use K-Means to group their customers into K distinct segments (e.g., 'High-Value,' 'Budget-Conscious,' 'New Customers') based on their purchasing habits, income, and age[38].

Q3: Attempt any Three $(4\times3=12)$

a) Compare Linear Regression with Logistic Regression.

Feature	Linear Regression	Logistic Regression	Aligned CO
Type of Problem	**Regression:** Used to predict a **continuous** output variable (e.g., house price, temperature)[39].	**Classification:** Used to predict a **categorical** output variable (e.g., spam/not spam, yes/no)[40].	CO3
Output Range	**$(-\infty, \infty)$** (any real number).	**$(0, 1)$**, representing a probability. This probability is then converted to a class label using a threshold (e.g., 0.5)[41].	CO3
Core Function	Uses a linear equation: $y = \beta_0 + \beta_1 x_1 + \dots$	Uses the **Sigmoid (or Logistic) function** to map the linear output to a probability between 0 and 1[42].	CO3
Model Evaluation	MSE, R-squared, MAE[43].	Confusion Matrix, Precision, Recall, F1-score, Accuracy[44].	CO3

b) Compare Grid Search and Random Search for hyperparameter tuning of model prediction.

Feature	Grid Search (GridSearchCV)	Random Search (RandomizedSearchCV)	Aligned CO
Mechanism	It exhaustively searches over a **specified subset**	It samples a **fixed number** of hyperparameter combinations	CO5

Feature	Grid Search (GridSearchCV)	Random Search (RandomizedSearchCV)	Aligned CO
	of hyperparameter values[45]. It tries **every single combination** of the defined parameter space.	from a specified distribution/space[46].	
Efficiency	Computationally **expensive** as the number of combinations grows exponentially with the number of hyperparameters.	Computationally **efficient** as it doesn't explore the entire space, often finding a good combination faster.	CO5
Effectiveness	**Guaranteed** to find the optimal combination within the defined grid, but might miss better values outside the grid.	**Not guaranteed** to find the absolute optimum, but often finds a very good solution, especially if many hyperparameters are irrelevant.	CO5
When to use	Small parameter spaces, or when you are confident the optimal values are in a specific, small range.	Large parameter spaces, or when you want to quickly narrow down the best regions of the space.	CO5

c) Write a Python Program to calculate MSE and R^{2} for given values.

Given values:

| Actual (Y) | Predicted ($\hat{Y}$) |

| :--- | :--- |

| 15 | 14 |

| 18 | 20 |

| 10 | 11 |

| 12 | 8 |

Python
```
import numpy as np
from sklearn.metrics import mean_squared_error, r2_score

# Given Actual and Predicted values
```

```
Y_actual = np.array([15, 18, 10, 12])
Y_predicted = np.array([14, 20, 11, 8])

print("Actual Values (Y):", Y_actual)
print("Predicted Values (Y_hat):", Y_predicted)

# 1. Calculate Mean Squared Error (MSE)
# Formula: MSE = (1/n) * Σ(Y - Y_hat)^2
mse_val = mean_squared_error(Y_actual, Y_predicted)

# 2. Calculate R-squared (R2)
# Formula: R2 = 1 - (SSE / SST)
r2_val = r2_score(Y_actual, Y_predicted)

print("\n--- Model Evaluation Metrics ---")
print(f"Mean Squared Error (MSE): {mse_val:.4f}")
print(f"R-squared (R^2) Score: {r2_val:.4f}")

# Manual Calculation for understanding:
errors = Y_actual - Y_predicted  # [1, -2, -1, 4]
squared_errors = errors**2        # [1, 4, 1, 16]
sum_squared_errors = np.sum(squared_errors) # 22
manual_mse = sum_squared_errors / len(Y_actual) # 22 / 4 = 5.5
# print(f"Manual MSE: {manual_mse}")
```

d) Compare Bias and Variance of Machine Learning.

The concepts of Bias and Variance describe the two main sources of error that can affect a machine learning model's ability to generalize to new, unseen data, known as the Bias-Variance Trade-off47.

| Feature | Bias | Variance | Aligned CO |

| :--- | :--- | :--- | :--- |

| Definition | An error introduced by approximating a real-world problem with a simplified model48. It is the difference between the expected prediction of our model and the true value49. | The error introduced due to the model's sensitivity to small fluctuations in the training data50. It is how much the predictions for a given point vary between different training datasets51. | CO6 |

| Effect | Causes the model to underfit the data (oversimplified)52. | Causes the model to overfit the data (too complex)53. | CO6 |

| Model Type | Typical of simple models (e.g., Linear Regression). | Typical of complex models (e.g., deep Decision Trees, k-NN with small k). | CO6 |

| Goal | We aim for low bias. | We aim for low variance. | CO6 |

e) Compare the Decision Tree with the Random Forest.

Feature	Decision Tree	Random Forest	Aligned CO
Model Type	A **single** tree structure[54].	An **Ensemble** method that uses a collection (forest) of multiple Decision Trees[55].	CO3
Training	Trains one tree on the entire training dataset.	Trains multiple trees, each on a **random subset** of the data and a random subset of features (bootstrap aggregating or bagging)[56].	CO3
Prediction	Prediction is made by traversing the single tree from the root to a leaf node.	Prediction is made by taking the **average** of the predictions (for regression) or the **majority vote** (for classification) from all the individual trees[57].	CO3
Bias/Variance	Generally has **low bias** but **high variance** (prone to overfitting)[58].	Generally has slightly **higher bias** than a single tree, but **significantly lower variance** (more robust and less prone to overfitting).	CO3

Q4: Attempt any Three $(4\times3=12)$

a) Compare overfitting and underfitting in machine learning models.

Overfitting and Underfitting describe common problems where a model fails to generalize properly to new data595959.

| Feature | Overfitting | Underfitting | Aligned CO |

| :--- | :--- | :--- | :--- |

| Training Error | Very low or near zero. | High (poor performance). | CO2 |

| Test/Generalization Error | High (much higher than training error). | High (similar to training error). | CO2 |

| Model Complexity | Too complex relative to the data6060. | Too simple relative to the data6161. | CO2 |

| Description | The model learns the noise and random fluctuations in the training data rather than the underlying pattern6262. | The model fails to capture the underlying pattern of the data and cannot learn the relationship between features and target6363. | CO2 |

| Solution | Reduce complexity (e.g., simplify model, use regularization, increase training data, use cross-validation)64. | Increase complexity (e.g., use a more powerful model, add more features, reduce regularization). | CO2 |

b) Describe the K-fold cross-validation technique for model evaluation.

K-fold cross-validation is a robust technique for **model evaluation** that helps to estimate how well a model will generalize to an independent dataset[65][65].

1. **Splitting:** The entire dataset is divided into K equal-sized, non-overlapping subsets, or "folds"[66].
2. **Iteration:** The process is repeated K times (or K "folds")[67].
   - In each iteration, **one fold** is designated as the **test/validation set**[68].
   - The **remaining $K-1$ folds** are combined to form the **training set**[69].
3. **Training and Evaluation:** The model is trained on the training set and evaluated on the test set. The performance metric (e.g., accuracy, MSE) is recorded[70].
4. **Final Result:** The **K recorded scores are averaged** to produce a single, final estimate of the model's performance[71].

Importance: It ensures that every data point gets to be in a test set exactly once, and every fold gets to be used for training, resulting in a more reliable and less biased estimate of the model's performance compared to a simple train/test split.

c) Write a program to perform Customer segmentation using the K-Means Algorithm. (Assume suitable data).

This program demonstrates K-Means for customer segmentation, a compulsory practical for CO4[72][72].

Python

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Assume Suitable Data: 'Spending' (Annual $) and 'Age' for 10
customers
data = {
    'Customer_ID': range(1, 11),
    'Age': [22, 55, 30, 25, 60, 45, 35, 50, 40, 28],
    'Annual_Spending': [500, 1500, 750, 600, 2000, 1200, 800, 1800,
1000, 700]
}
df = pd.DataFrame(data)

# Selecting features for clustering (Age and Spending)
X = df[['Age', 'Annual_Spending']].values

# Define the number of clusters (K). Let's choose K=3 for this example.
K = 3
kmeans = KMeans(n_clusters=K, random_state=42, n_init=10)

# Train the K-Means model
df['Cluster'] = kmeans.fit_predict(X)
```

```
print("--- Customer Segmentation Results (K=3) ---")
print(df)
print("\nCluster Centroids:\n", kmeans.cluster_centers_)

# Visualize the clusters plt.figure(figsize=(8, 6))
# Create a scatter plot for each cluster
for cluster in range(K):
    clustered_data = df[df['Cluster'] == cluster]
    plt.scatter(clustered_data['Age'],
clustered_data['Annual_Spending'],
                label=f'Cluster {cluster}', s=100) # s=size

# Plot the centroids
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:,
1],
            marker='X', s=200, c='red', label='Centroids') # X=Centroid
marker

plt.title('Customer Segmentation using K-Means (K=3)')
plt.xlabel('Age')
plt.ylabel('Annual Spending ($)')
plt.legend()
plt.grid(True)
plt.show()
```

d) Write a Python code to represent the given data using one-hot encoding.

Data: `data = {'Fruit': ['Apple', 'Banana', 'Cherry', 'Apple', 'Cherry', 'Banana']}` [73]

Python

```
import pandas as pd

# Given data
data = {'Fruit': ['Apple', 'Banana', 'Cherry', 'Apple', 'Cherry',
'Banana']}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Perform One-Hot Encoding using pandas get_dummies()
# The 'Fruit' column will be transformed into new binary columns.
df_encoded = pd.get_dummies(df, columns=['Fruit'])

print("\nOne-Hot Encoded DataFrame:\n", df_encoded)

# Note: The output will have 1 in the column corresponding to the
fruit, and 0 otherwise.
# A small excerpt of the output:
#    Fruit_Apple  Fruit_Banana  Fruit_Cherry
# 0            1             0             0
# 1            0             1             0
# 2            0             0             1
```

e) Describe standardisation and Normalisation techniques of feature scaling with an example.

Feature Scaling is a step in data preprocessing that standardizes or normalizes the range of independent features (variables)[74][74].

1. Normalisation (Min-Max Scaling)

- **Description:** Normalization scales the data to a fixed range, usually **[0, 1]**[75]. This is done by subtracting the minimum value and dividing by the range (max - min)[76].
- Formula:

  $$X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Example:** A feature representing `Age` with values from 18 to 68.
  - An Age of 38 would be normalized to: $\frac{38 - 18}{68 - 18} = \frac{20}{50} = 0.4$
- **When to use:** Used when the data distribution is **not Gaussian** or when algorithms like K-Nearest Neighbors (k-NN) and Neural Networks are used[77]. It is sensitive to outliers.

2. Standardisation (Z-Score Normalisation)

- **Description:** Standardisation scales data to have a **mean (μ) of 0** and a **standard deviation (σ) of 1**[78][78][78]. This is done by subtracting the mean and dividing by the standard deviation[79].
- Formula:

  $$X_{standardized} = \frac{X - \mu}{\sigma}$$

- **Example:** A feature with a mean of 50 and standard deviation of 10.
  - A value of 70 would be standardized to: $\frac{70 - 50}{10} = \frac{20}{10} = 2.0$
- **When to use:** Used when the data distribution is **Gaussian (bell curve)**, or for algorithms that assume normally distributed data (e.g., Linear Regression, Logistic Regression)[80]. It is less affected by outliers than Min-Max Normalisation.

Q5: Attempt any Two $(6\times2=12)$

a) Write a program to implement PCA on the Iris dataset (4 features) to reduce it to 2 components and visualise the results.

This is a key practical for **Dimensionality Reduction (PCA)** from CO4[81].
Python
```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

# 1. Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
```

```
target_names = iris.target_names
print(f"Original Data Shape: {X.shape}") # (150, 4)

# 2. Standardize the data (PCA is sensitive to scale)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Apply PCA: Reduce to 2 components
# n_components=2 is for 2D visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print(f"Reduced Data Shape: {X_pca.shape}") # (150, 2)

# Create a DataFrame for the reduced data for easy plotting
pca_df = pd.DataFrame(data = X_pca, columns = ['principal_component_1',
'principal_component_2'])
pca_df['target'] = y

# 4. Visualize the results plt.figure(figsize=(8, 8))
colors = ['r', 'g', 'b']
for target, color in zip(np.unique(y), colors):
    # Filter the data for the current target (species)
    indicesToKeep = pca_df['target'] == target
    plt.scatter(pca_df.loc[indicesToKeep, 'principal_component_1'],
                pca_df.loc[indicesToKeep, 'principal_component_2'],
                c = color,
                s = 50,
                label = target_names[target])

plt.xlabel('Principal Component 1 (PC1)', fontsize = 15)
plt.ylabel('Principal Component 2 (PC2)', fontsize = 15)
plt.title('2 Component PCA of Iris Dataset', fontsize = 20)
plt.legend(target_names)
plt.grid()
plt.show()

# Optional: Explained variance ratio
explained_variance = pca.explained_variance_ratio_
print(f"\nExplained Variance Ratio of PC1:
{explained_variance[0]:.4f}")
print(f"Explained Variance Ratio of PC2: {explained_variance[1]:.4f}")
print(f"Total Variance Explained by 2 components:
{np.sum(explained_variance):.4f}")
```

b) Solve the given problem mathematically and predict marks for 6 study hours (Linear Regression).

Data:

| Study Hours (X) | Exam Marks (Y) |

| :--- | :--- |

| 2 | 50 |

| 3 | 60 |

| 5 | 75 |

| 7 | 85 |

| 9 | 95 |

The problem requires a **Linear Regression** model: $Y = mX + c$, where m is the slope and c is the intercept.

1. Calculate Slope (m) and Intercept (c)

X	**Y**	**X2**	**XY**
2	50	4	100
3	60	9	180
5	75	25	375
7	85	49	595
9	95	81	855
Σ	**$\Sigma X=26$**	**$\Sigma Y=365$**	**$\Sigma X^2=168$**

Number of data points, $n = 5$.

- **Means:**
  - $\bar{X} = \Sigma X / n = 26 / 5 = 5.2$
  - $\bar{Y} = \Sigma Y / n = 365 / 5 = 73$
- Slope (m):

  $$m = \frac{n(\Sigma XY) - (\Sigma X)(\Sigma Y)}{n(\Sigma X^2) - (\Sigma X)^2}$$

  $$m = \frac{5(2105) - (26)(365)}{5(168) - (26)^2}$$

  $$m = \frac{10525 - 9490}{840 - 676} = \frac{1035}{164} \approx **6.311**$$

- Intercept (c):

  $$c = \bar{Y} - m\bar{X}$$

  $$c = 73 - (6.311)(5.2)$$

  $$c = 73 - 32.8172 \approx **40.183**$$

- **Regression Equation:** $Y = 6.311X + 40.183$

  2. Predict marks for students who study 6 hours.

Substitute $X=6$ into the regression equation:

$$Y_{predict} = 6.311(6) + 40.183$$
$$Y_{predict} = 37.866 + 40.183 \approx **78.049 \text{ Marks}**$$

3. Write a Python program for the above (Linear Regression).
Python
```
import numpy as np
from sklearn.linear_model import LinearRegression

# Given Data
X = np.array([2, 3, 5, 7, 9]).reshape(-1, 1) # Study Hours (Feature)
Y = np.array([50, 60, 75, 85, 95])           # Exam Marks (Target)

# 1. Train the Linear Regression Model
model = LinearRegression()
model.fit(X, Y)

# Get the calculated slope (m) and intercept (c)
slope = model.coef_[0]
intercept = model.intercept_

print("--- Linear Regression Results ---")
print(f"1. Calculated Slope (m): {slope:.4f}")
print(f"   Calculated Intercept (c): {intercept:.4f}")
print(f"   Regression Equation: Y = {slope:.4f} * X + {intercept:.4f}")

# 2. Predict marks for 6 study hours
X_new = np.array([[6]])
Y_predict_6hrs = model.predict(X_new)[0]

print(f"\n2. Predicted Marks for 6 study hours: {Y_predict_6hrs:.3f}")
```

c) Write a program to tune hyperparameters using Grid search and Random search which is going to train on Decision tree classifier.

This program applies the CO5 practical, using the given data and a Decision Tree Classifier[82].
Python
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV,
RandomizedSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Given Data
data = {
    'Age': [25, 30, 45, 22, 35, 50, 40, 29, 55, 42],
    'Income': [50000, 60000, 80000, 45000, 75000, 90000, 82000, 54000,
100000, 85000],
    'Loan Amount': [2000, 4000, 10000, 1500, 5000, 12000, 9000, 3000,
15000, 11000],
    'Credit_Score': [650, 700, 600, 720, 710, 580, 690, 660, 560, 640],
    'Default': ['No', 'No', 'Yes', 'No', 'No', 'Yes', 'No', 'No',
'Yes', 'Yes']
}
```

```
df = pd.DataFrame(data)

# Preprocessing: Convert 'Default' to numerical (0/1)
df['Default'] = df['Default'].apply(lambda x: 1 if x == 'Yes' else 0)

# Define Features (X) and Target (y)
X = df.drop('Default', axis=1)
y = df['Default']

# Scale numerical features (important for many ML models, though less
so for Decision Tree)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data (Train and Test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.3, random_state=42)

# Define the parameter space for the Decision Tree Classifier
# Common hyperparameters for Decision Tree:
param_grid = {
    'max_depth': [3, 5, 7, 9, 11, None],
    'min_samples_split': [2, 5, 10],
    'criterion': ['gini', 'entropy']
}

# 1. --- Grid Search ---
print("--- Starting Grid Search ---")
dtc = DecisionTreeClassifier(random_state=42)
grid_search = GridSearchCV(estimator=dtc, param_grid=param_grid, cv=3,
scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

best_grid_model = grid_search.best_estimator_
grid_accuracy = accuracy_score(y_test, best_grid_model.predict(X_test))

print(f"Grid Search Best Hyperparameters: {grid_search.best_params_}")
print(f"Grid Search Best Training Score:
{grid_search.best_score_:.4f}")
print(f"Grid Search Test Accuracy: {grid_accuracy:.4f}")

# 2. --- Random Search ---
print("\n--- Starting Random Search ---")
# Only search 10 random combinations (n_iter=10)
random_search = RandomizedSearchCV(estimator=dtc,
param_distributions=param_grid,
                                   n_iter=10, cv=3, scoring='accuracy',
random_state=42, n_jobs=-1)
random_search.fit(X_train, y_train)

best_random_model = random_search.best_estimator_
random_accuracy = accuracy_score(y_test,
best_random_model.predict(X_test))

print(f"Random Search Best Hyperparameters:
{random_search.best_params_}")
print(f"Random Search Best Training Score:
{random_search.best_score_:.4f}")
```

```
print(f"Random Search Test Accuracy: {random_accuracy:.4f}")
```

Q6: Attempt any Two $(6\times2=12)$

a) Write a program to Split given tabular data into train, test and validation.

This addresses the CO5 practical on data splitting[83].

Data: A sample tabular dataset with columns ID, Age, Income, and Purchased.
Python
```
import pandas as pd
from sklearn.model_selection import train_test_split

# Given Data (Tabular data)
data = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Age': [22, 35, 47, 52, 46, 56, 55, 60, 23, 40],
    'Income': [35000, 40000, 52000, 60000, 48000, 70000, 71000, 65000,
35000, 45000],
    'Purchased': ['No', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No',
'Yes', 'Yes']
}
df = pd.DataFrame(data)

# Separate features (X) and target (y)
X = df[['Age', 'Income']]
y = df['Purchased']

# 1. Split into Training Set and Temp Set (80% Train, 20% Temp)
# temp_size = 0.2 means the remaining 20% is split between test and
validation.
# We will use 0.6 for training, and split the remaining 0.4 into 0.2
for test and 0.2 for validation.
# Let's target 60% Train, 20% Validation, 20% Test (0.6 / 0.2 / 0.2)

# Step 1: Split into Train (60%) and Temp (40%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y,
test_size=0.4, random_state=42)

# Step 2: Split Temp (40%) into Validation (50% of Temp = 20% of total)
and Test (50% of Temp = 20% of total)
# 0.5 because X_temp is 40% of the original data. 0.5 * 0.4 = 0.2 (20%
of total)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
test_size=0.5, random_state=42)

print("--- Data Splitting Results (60/20/20) ---")
print(f"Original Data size: {len(X)}")
print(f"Training Set size (60%): {len(X_train)} (X_train shape:
{X_train.shape})")
print(f"Validation Set size (20%): {len(X_val)} (X_val shape:
{X_val.shape})")
print(f"Test Set size (20%): {len(X_test)} (X_test shape:
{X_test.shape})")
```

b) Write a program to represent given data using Hashing encoding technique.

Data: `data = [{'fruit': 'apple'}, {'fruit': 'banana'}, {'fruit': 'cherry'}, {'fruit': 'apple'}, {'fruit': 'banana'}]`

Hashing Encoding, or Feature Hashing, converts categorical features into numerical ones using a hash function. It has a fixed output space size, which helps manage high-cardinality features and avoids increasing dimensionality.

Python
```
import pandas as pd
from sklearn.feature_extraction import FeatureHasher

# Given Data
data = [{'fruit': 'apple'}, {'fruit': 'banana'}, {'fruit': 'cherry'}, {'fruit': 'apple'}, {'fruit': 'banana'}]
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Initialize FeatureHasher
# n_features: The number of features in the output (fixed size)
# input_type='dict': Since the input data is a list of dictionaries
h = FeatureHasher(n_features=4, input_type='dict', non_negative=True)

# Fit and transform the data
# The transform method expects an iterable of dicts.
hashed_features = h.fit_transform(df.to_dict('records'))

# Convert the sparse matrix output to a dense array for printing
hashed_features_array = hashed_features.toarray()

# Create a DataFrame for the hashed features for better presentation
hashed_df = pd.DataFrame(hashed_features_array,
                         columns=[f'Hash_Col_{i+1}' for i in range(hashed_features_array.shape[1])])

print("\n--- Hashing Encoded Result (n_features=4) ---")
print(hashed_df)

# Note: The 'fruit' values ('apple', 'banana', 'cherry') are mapped
# to one of the 4 hash columns. Collisions (different values mapping to
# the same column) are possible and are summed up in the resulting column.
```

c) Solve the given Logistic regression problem mathematically.

Problem: The provided table is suitable for solving a **Logistic Regression** problem, which is a key supervised learning algorithm from CO3[84]. Since only the data is given, we must assume we are trying to predict the **Spam** status (1 for Yes, 0 for No) based on the **"FREE" Count** and **"Links" Count**.

Email	"FREE" Count (X1)	"Links" Count (X2)	Spam (Y)

Email	"FREE" Count (X1)	"Links" Count (X2)	Spam (Y)
1	5	1	1
2	3	2	0
3	0	0	1
4	7	1	0
5	2	4	0
6	5	5	1

The problem is to predict the probability of $Y=1$ (Spam) using the Logistic Regression model.

Assumptions for Mathematical Solution:

Since the coefficients ($\beta_0, \beta_1, \beta_2$) are **not provided**, we must **assume a set of coefficients** to demonstrate the *mathematical calculation* of the probability, as solving for the coefficients (Maximum Likelihood Estimation) is computationally complex.

Assume Coeffiecients:

- **Intercept $\beta_0 = -1.5$**
- **Coefficient for "FREE" Count $\beta_1 = 0.5$**
- **Coefficient for "Links" Count $\beta_2 = 0.2$**

Logistic Regression Equation:

The probability $P(Y=1)$ is given by the Sigmoid function:

$$P(Y=1 \mid X) = \frac{1}{1 + e^{-z}}$$

where z is the linear combination of features:

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Consider Email 1: $X_1=5$, $X_2=1$. ($Y=1$ is the actual outcome)

1. Calculate z (Logit):

   $$z = -1.5 + 0.5(5) + 0.2(1)$$

   $$z = -1.5 + 2.5 + 0.2$$

   $$z = **1.2**$$

2. Calculate Probability $P(Y=1)$ (Sigmoid):

   (Given $e \approx 2.71828$)

   $$P(Y=1) = \frac{1}{1 + e^{-1.2}}$$

   $$P(Y=1) = \frac{1}{1 + (2.71828)^{-1.2}}$$

   $$P(Y=1) \approx \frac{1}{1 + 0.30119} \approx \frac{1}{1.30119} \approx **0.7685**$$

3. **Apply Threshold:**
   - **Threshold:** 0.5 (Given)
   - Since $0.7685 > 0.5$, the model **predicts $Y=1$ (Spam)**.

- *(Since the actual Y is also 1, this is a **True Positive**.)*

  Conclusion for Email 1: Based on the assumed coefficients, the probability of Email 1 being spam is approximately $\mathbf{0.7685}$. Applying the $\mathbf{0.5}$ threshold, the email is **predicted as Spam**.